



H2S

BHARATIYA ANTARIKSH HACKATHON 2026



Team Name : NeuralNinjas

Team Leader Name : Sandeep

Problem Statement : 13 - Air-Gapped Predictive Copilot for Secure MPLS Operations

Team Members

Team Leader:

Name: Sandeep
College: Vemana Institute of
Technology

Team Member-1:

Name: Alwin Joy
College: Vemana Institute of
Technology

Team Member-2:

Name: Subharup Nandi
College: Vemana Institute of
Technology

AirGuard AI — Why This is Different & Why It Will Work

- **How it differs:** Existing NOC tools (SolarWinds, Zabbix, Nagios) are purely reactive they alert after breach. AirGuard AI detects statistical precursors 15–30 minutes before SLA impact using time-series forecasting and graph anomaly detection. No existing tool combines predictive ML + offline LLM + digital twin in one air-gapped package.
- **How it solves the problem:** Telemetry from SNMP/NetFlow/Syslog feeds an LSTM forecaster (utilization, latency, jitter) and an XGBoost classifier (fault type). A NetworkX graph engine propagates risk scores across topology. A quantised Phi-3 Mini LLM (4-bit GGUF, ~4GB RAM) generates operator-ready explanations via RAG over local runbooks and incident history all fully offline, zero cloud dependency.
- **USP 1 — Mission Impact Prediction:** Unlike device-level alerts, AirGuard AI maps network failures to operational consequences (e.g., MPLS congestion → ground station telemetry delay → satellite data processing backlog). Operators understand real-world impact, not just raw metrics.
- **USP 2 — What-If Simulation Engine:** Operators query “What if I reroute 40% of traffic to backup tunnel T2?” The system clones the live digital twin, replays modified traffic matrices through the LSTM, and returns predicted latency delta, SLA status, and recovery time estimate enabling safe, pre-validated remediation decisions.

Key Features of AirGuard AI

- **Predictive Fault Forecasting:** LSTM models trained on SNMP time-series predict congestion, latency drift, and interface saturation 15–30 min ahead with target precision >85%.
- **Route Instability Detection:** XGBoost classifier detects BGP flap precursors and OSPF convergence stress from routing event logs before path instability cascades.
- **Graph-Based Risk Propagation:** NetworkX digital twin propagates link risk scores across CE/PE/P topology to identify downstream impact zones before failures materialize.
- **Offline LLM Copilot (RAG):** Phi-3 Mini (4-bit GGUF) + ChromaDB + all-MiniLM-L6-v2 embeddings. Runbooks, incident history, and topology metadata chunked and indexed locally. LLM retrieves top-k context and generates structured operator responses in <4 seconds on CPU.
- **Mission Impact Prediction:** Maps network fault predictions to operational service consequences (e.g., MPLS congestion → telemetry transfer delay → ground station sync degradation) enabling priority-based NOC response.
- **What-If Simulation Engine:** Clones live digital twin state, modifies traffic matrix per operator query, re-runs LSTM inference on modified input, and returns predicted latency delta + SLA status within seconds.
- **Confidence-Scored Alert Prioritization:** Composite risk scores combining model confidence, time-to-impact, and affected service criticality suppress noise and surface only actionable alerts. Target false positive rate <5%.

End-to-End Process Flow — Fault Scenario Walkthrough

STEP 1 — Telemetry Ingestion: Telegraf polls SNMP every 30s from CE/PE/P routers in Containerlab topology. NetFlow records, BGP/OSPF syslog events, tunnel stats, and controller state changes are streamed into InfluxDB time-series store.

STEP 2 — Feature Engineering: Raw counters are transformed into 60-second sliding window features: utilization%, delta_utilization, jitter_std, packet_loss_rate, bgp_flap_count_5min, ospf_isa_flood_rate. Missing values imputed with forward-fill. Labels assigned from injected fault ground-truth timestamps.

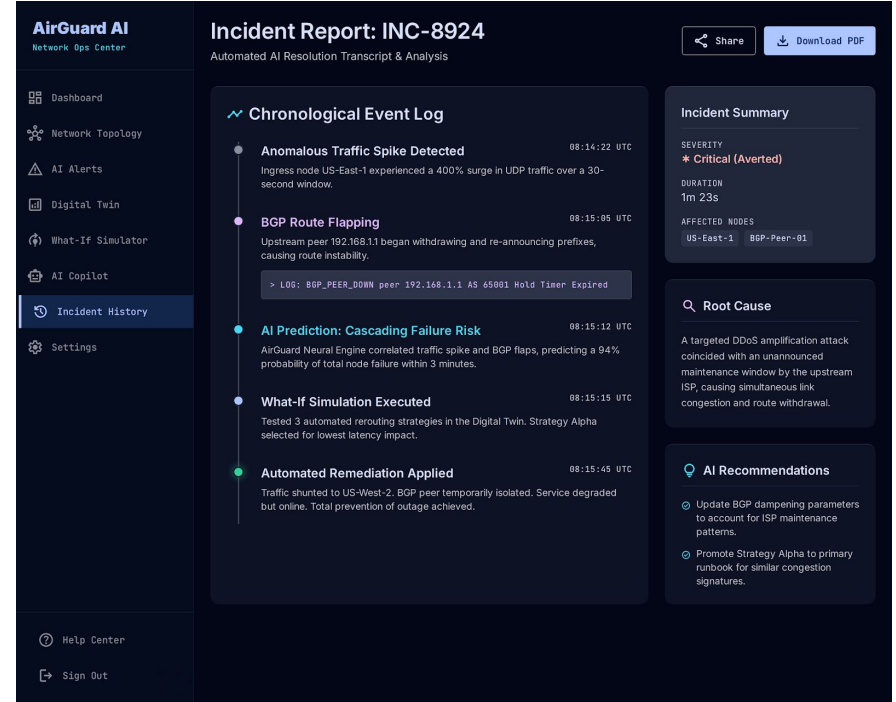
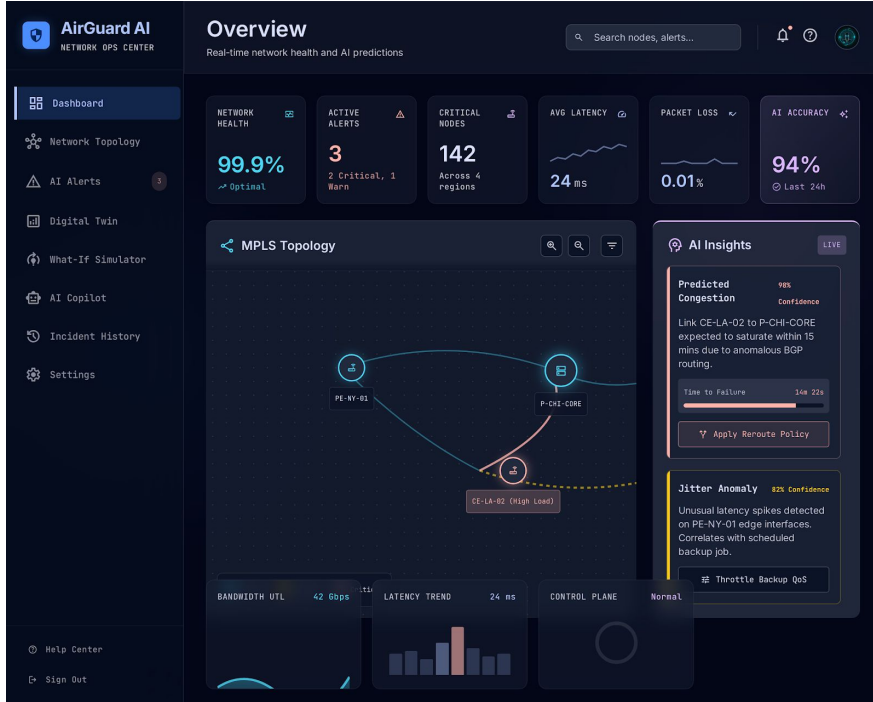
STEP 3 — Prediction Engine: LSTM (sequence length=20, hidden=128, 2 layers) forecasts utilization and latency 15–30 min ahead. XGBoost classifier (150 estimators, max_depth=6, SMOTE-balanced) predicts fault type: {congestion, bgp_flap, tunnel_degradation, policy_drift}. Outputs: fault_type, confidence_score, time_to_impact_minutes.

STEP 4 — Graph Risk Propagation: NetworkX graph of CE/PE/P topology updated every 60s. Link risk scores from LSTM propagated to downstream nodes using weighted adjacency traversal. Affected sites, VPNs, and services identified automatically.

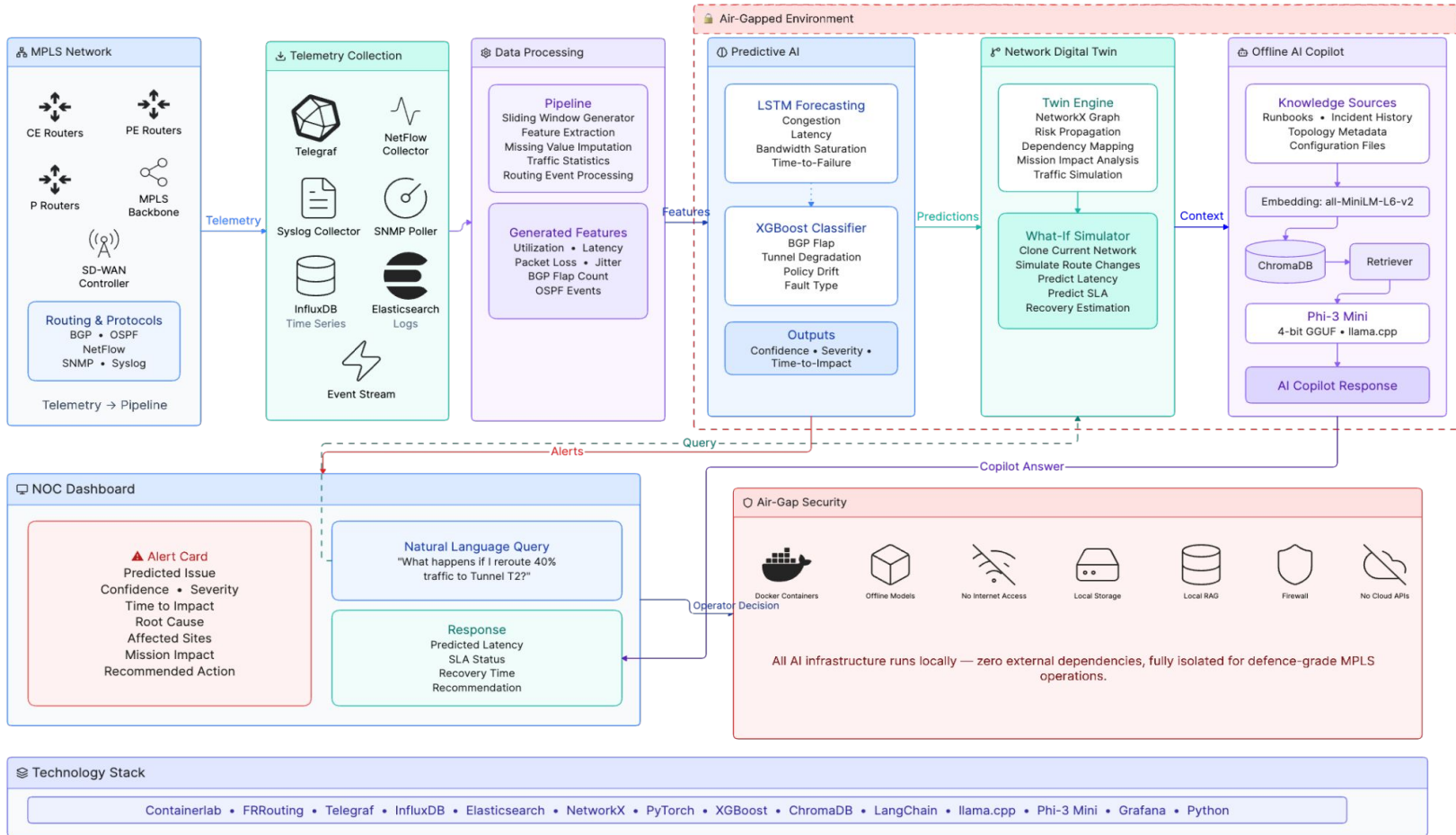
STEP 5 — RAG + LLM Response: Alert context (fault_type, confidence, affected nodes, time_to_impact) embedded with all-MiniLM-L6-v2. ChromaDB retrieves top-5 relevant runbook chunks + incident history entries. Phi-3 Mini generates: Predicted Issue | Confidence | Root Cause | Affected Scope | Recommended Actions.

EXAMPLE OUTPUT — Congestion Scenario: *“ALERT [HIGH | 87% confidence]: PE2-HUB1 link utilization at 78% and trending toward saturation. Estimated time to SLA breach: 23 minutes. Affected: Branch-3, Branch-4, VPN-MGMT. Root cause: sustained burst traffic from datacenter backup job. Recommended: Redistribute 35% of Branch-3 traffic to backup IPsec tunnel T2. Estimated recovery: latency -32%, congestion eliminated.”*

Wireframes/Mock diagrams of the proposed solution



Architecture Diagram



Technologies to be used in the solution:

- **Network Simulation:** Containerlab (topology orchestration), FRRouting (BGP/OSPF/MPLS), D-ITG (traffic generation), Python fault injection scripts
- **Telemetry Pipeline:** Telegraf (SNMP/syslog collection), InfluxDB (time-series), Elasticsearch (event indexing), nfdump (NetFlow), Grafana (visualization dashboard)
- **Digital Twin & Graph:** NetworkX (topology graph + risk propagation), Python (digital twin state management, What-If simulation engine)
- **Predictive ML:** PyTorch (LSTM — seq=20, hidden=128, 2 layers), XGBoost (fault classifier, SMOTE-balanced, 150 estimators, max_depth=6), Prophet (trend decomposition), scikit-learn (preprocessing, evaluation)
- **Offline LLM + RAG:** Phi-3 Mini 3.8B Q4_K_M GGUF via llama.cpp (CPU inference ~2–4s), ChromaDB (local vector DB), sentence-transformers all-MiniLM-L6-v2 (local embeddings, no internet), LangChain (RAG orchestration, fully offline)
- **Air-Gap Security:** All components containerized and pre-bundled (Docker Compose offline deployment). No outbound network calls at runtime. Verified using iptables egress filtering. All model weights, embeddings, and dependencies shipped in offline bundle.

Estimated implementation cost:

Prototype Development:

- All components open-source: Containerlab, FRRouting, Telegraf, InfluxDB, ChromaDB, PyTorch, XGBoost, llama.cpp, Phi-3 Mini — ₹0 licensing cost
- Hardware: Existing development machines (16GB RAM, 8-core CPU sufficient for full prototype). No GPU required for Phi-3 inference on GGUF quantized model. Estimated total prototype cost: ₹0 – ₹5,000

VALIDATION SCENARIOS (Phase 6):

- **S1 — Progressive Congestion:** Gradual traffic ramp on hub-spoke link. Target: LSTM predicts saturation 15–30 min ahead with >85% precision, false positive rate <5%.
- **S2 — BGP Route Flap:** Inject BGP session instability leading to path reroute cascade. XGBoost must detect flap precursors (flap_count + hold-timer warnings) before convergence failure, with correct root cause classification.
- **S3 — MPLS Underlay Failure:** Intermittent P-router link failure causing tunnel packet loss progression. Tunnel health scoring must detect degradation trend before complete loss, and copilot must recommend correct failover path.
- **S4 — Policy Drift:** SD-WAN controller misconfiguration causing QoS policy inconsistency. System must detect anomalous traffic class distribution and controller state divergence before service impact.

BHARATIYA
ANTARIKSH
HACKATHON

2026

THANK YOU